

Tablas para control del flujo de los hoteles

reglas claras:

- **customers** = “cliente/billing” (quién paga / Stripe / datos fiscales)
- **debacu_eval_organizations** = “tenant” (el hotel/empresa dentro de Debacu Eval)
- **debacu_eval_org_members** = “permisos” (usuarios que entran a ese tenant)
- **debacu_eval_hotel_profile** = “configuración operativa” (ADR, estacionalidad, etc.)

```
usaremos update debacu_eval_organizations
set customer_id = '059a0113-ff49-49af-9be8-77d97a9ef866'
where id = 'b5909245-9bb5-488e-a7f2-4fb4762922a1';
```

Guía de arquitectura y flujo Debacu Eval (Documento vivo)

0) Definiciones (las que tú marcas)

- **customers** = *cliente/billing*
Quién paga. Datos fiscales. Stripe customer/subscription. Puede pagar por 1 o varios tenants según el futuro, pero ahora 1:1 está bien.
- **debacu_eval_organizations** = *tenant*
El “hotel/empresa” dentro de Debacu Eval (ámbito de datos y RLS).
- **debacu_eval_org_members** = *permisos*
Usuarios (auth.users) que pueden entrar al tenant, con **role** y **status**.
- **debacu_eval_hotel_profile** = *configuración operativa*
Parámetros operativos: checkin/checkout, ADR base, estacionalidad, etc.
- **debacu_eval_properties** = *hoteles / centros / unidades operativas dentro del tenant*
Aquí es donde cuadra el multi-hotel por org sin romper el tenant.
En demo podrás tener 1 property por org, pero dejamos soportado N.

1) Regla de oro de multi-hotel (para no romper nada)

- **org_id** = **scope de seguridad y RLS** (lo que separa tenants).
- **property_id** = **scope operativo** (qué hotel dentro del tenant).
- Todas las tablas “de negocio” nuevas deben llevar:
 - **org_id** NOT NULL
 - **property_id** NULLABLE (de momento) → en producción lo haremos NOT NULL cuando el flujo esté cerrado.

En desarrollo: `property_id` puede ser null temporalmente; en nuevas escrituras intentaremos meterlo siempre desde Edge.

2) Tablas “core” y responsabilidades

2.1. Identidad y acceso

- `debacu_eval_organizations`
 - Crea el tenant.
- `debacu_eval_org_members`
 - Controla quién entra. OWNER/ADMIN/STAFF + ACTIVE/INVITED/ . . .
- (Opcional más adelante) `debacu_eval_customers` o tabla de unión customer ↔ org si necesitas 1 billing para varios orgs.

2.2. Operación hotelera

- `debacu_eval_properties`
 - Cada fila = un hotel (o propiedad) dentro del org.
- `debacu_eval_hotel_profile`
 - Config de operación (horas, reglas, etc.) **normalizada**.

2.3. Núcleo de datos de huéspedes (nuevo estándar)

A partir de ahora, para todo lo nuevo:

- `debacu_eval_guest_index` = “huésped” global del tenant (por `identity_key` hash)
- `debacu_eval_guest_stays` = “estancias / visitas / reservas” (una por estancia)
- `debacu_evaluations` = tu tabla legacy / histórico de incidencias (se mantiene, pero dejamos de ampliarla como eje central)

Idea simple:

- `guest_index` responde “¿quién es?”
 - `guest_stays` responde “¿cuándo vino y qué pasó?”
 - `evidences/incidents` responden “¿qué prueba o incidencia tengo?”
-

3) Flujo 1: Admin acepta solicitud de acceso (admin@debacu.com)

Objetivo

Cuando un hotel solicita acceso, **admin ejecuta un único flujo** que:

1. valida la solicitud
2. crea tenant y estructura mínima
3. crea permisos/membership
4. envía invitación al email del hotel para que cree password y entre

Artefactos que se crean (mínimo viable)

A) Tenant

- Insert en `debacu_eval_organizations`
 - `id, name, status=ACTIVE` (o `PENDING` si quieres), timestamps

B) Property “por defecto”

- Insert en `debacu_eval_properties`
 - `org_id, code (canon), name, timezone, is_active=true`
 - Regla: **1 property default** al crear el org (aunque luego haya más).

C) Perfil operativo

- Insert en `debacu_eval_hotel_profile`
 - `org_id, property_id` (si aplica), valores por defecto
 - Importante: check-in/out en formato fijo (te lo normalizo cuando me pases la Edge que falla)

D) Permisos

- Insert en `debacu_eval_org_members`
 - `org_id, auth_user_id` (del invitado si existe ya, si no por `invited_email`)
 - `role=OWNER` (para el hotel)
 - `status=INVITED`

E) Invitación

- Invitar por Supabase Auth Admin:
 - `auth.admin.inviteUserByEmail(email, { redirectTo: ... })`
 - En metadata puedes meter: `org_id, invited_role, property_id_default`

F) (Opcional) Enlace con billing

- Si ya tienes `customers` (billing) creado antes o durante la solicitud:
 - guarda `customer_id` o `creator_customer_id` en `org` o en una tabla de mapping.
-

4) Contratos de Edge Functions (normas)

Para que no se descontrolen:

4.1. Respuesta estándar

Siempre:

```
{ "ok": true, "data": ... }  
{ "ok": false, "error": { "code": "...", "message": "...", "details": ... } }
```

4.2. Seguridad

- Todas las Edge Functions: **JWT-only** (`requireUser`)
- Para admin: comprobación dura:
 - o bien `email == admin@debacu.com`
 - o un rol “ADMIN_SYSTEM” en una tabla de admins (mejor, pero en demo vale email hardcodeado)

4.3. Scope

- Cualquier insert/consulta de tenant requiere `org_id`.
 - Cualquier operación operativa requiere `property_id` (si ya existe selector) o se usa el default.
-

5) Plan de trabajo (cómo seguimos sin perdernos)

Vamos pantalla por pantalla.

Paso A (ahora): Admin aceptación acceso

- Me pasas:
 1. Screenshot de la pantalla admin
 2. Edge Function que usa esa pantalla (la primera)
 3. Tablas implicadas (si no las sé, con nombres basta)

Y yo te devuelvo:

- Flujo cerrado (SQL + Edge)
- Qué columnas faltan / sobran
- Qué RLS/políticas hay que tener para que funcione

- Qué queda “pendiente” para producción

Paso B: Hotel invitado crea contraseña y entra

- Pantalla activate / finalize + Edge(s)

Paso C: Perfil del hotel (hotel_profile) + normalización inputs

- Aquí resolvemos lo de check-in/out y “todo en mayúsculas” (lo hacemos en Edge o en DB con trigger; en demo mejor en Edge).

Paso D: Primeras pantallas de negocio (guest_index / guest_stays)

- Todo lo nuevo ya va a esas tablas.
-

6) Nota importante sobre tus datos demo actuales (para que no te explote)

- **No conviertas a NOT NULL property_id ahora.**
 - En demo puedes dejar debacu_evaluations como histórico “sucio”.
 - A partir del momento que creemos hotel1@ y hotel2@:
 - todo registro nuevo debe entrar ya por guest_index y guest_stays con org_id y property_id correctos.
-

Cuando quieras, pásame **la pantalla de admin** y **la primera Edge Function** (la que aprueba/acepta solicitud). A partir de ahí la dejamos “perfecta”: crea org + property + members + invite, y lo dejamos listo para duplicar para hotel1@ y hotel2@.